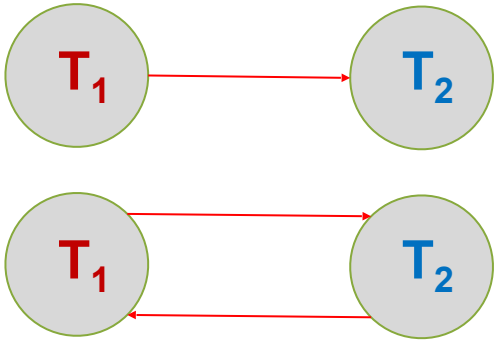
What can we say about "good" vs. "bad" conflict graphs?

Serial Schedule:

Simple!

Interleaved Schedules:



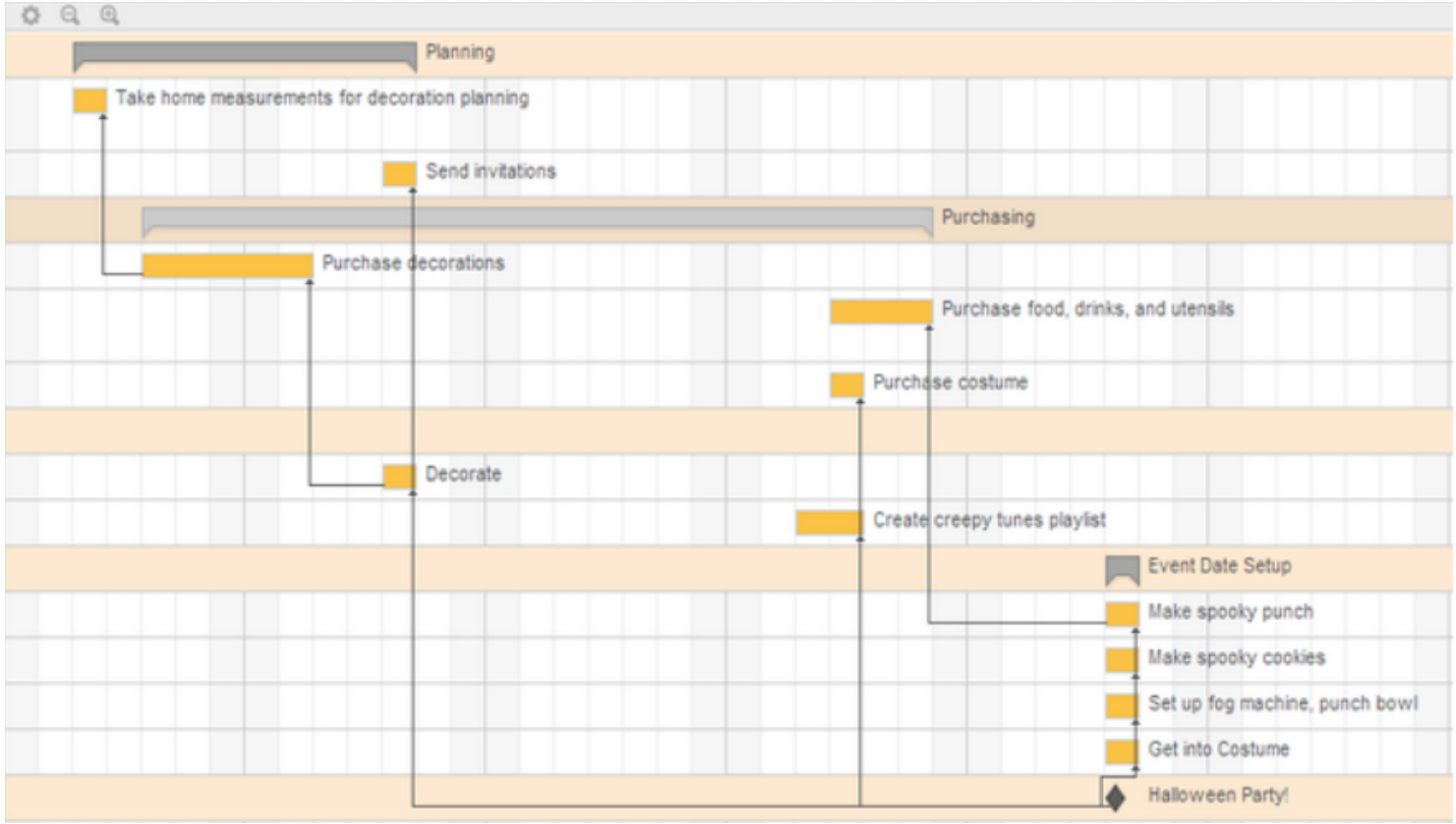
Theorem: Schedule is **conflict serializable** if and only if its conflict graph is **acyclic**

DAGs & Topological Orderings

- A **topological ordering** of a directed graph is a linear ordering of its vertices that respects all the directed edges
  - E.g., if vertex i has a directed edge to vertex j, in any topological ordering vertex i would appear before vertex j

- A directed **acyclic** graph (DAG) always has one or more **topological orderings**
  - (And there exists a topological ordering *if and only if* there are no directed cycles)

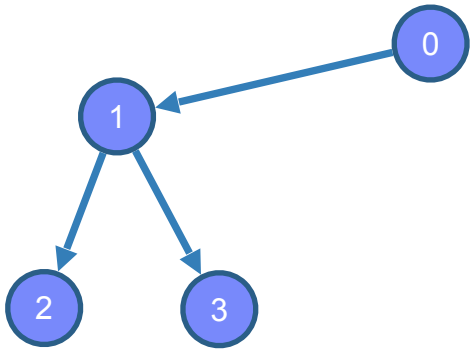
Example: Project dependencies



How would you plan?
What if there are cycles? (dependencies)

DAGs & Topological Orderings

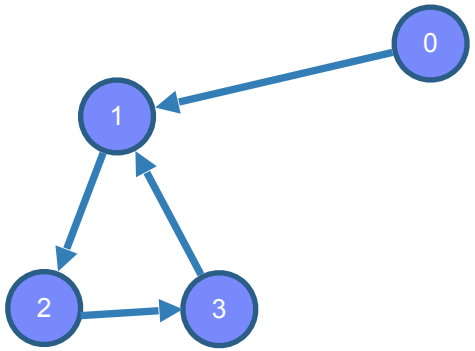
- Ex: What is one possible topological ordering here?



Ex: 0, 1, 2, 3 (or: 0, 1, 3, 2)

DAGs & Topological Orderings

- Ex: What is one possible topological ordering here?



There is none!

Connection to conflict serializability

- In the conflict graph, a topological ordering of nodes corresponds to **a serial ordering of TXNs**
- Thus an **acyclic** conflict graph → conflict serializable!

Theorem: Schedule is **conflict serializable** if and only if its conflict graph is **acyclic**

Example with 5 transactions

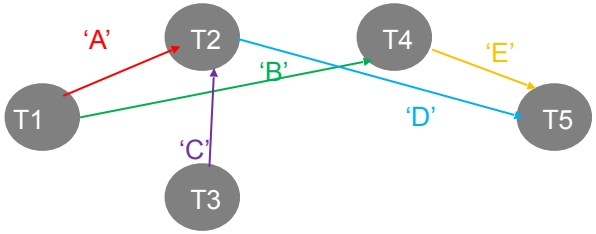
Schedule S1

Good or Bad schedule?
Conflict serializable?

	w1(A)	r2(A)	w1(B)	w3(C)	r2(C)	r4(B)	w2(D)	w4(E)	r5(D)	w5(E)
T1	w1(A)		w1(B)							
T2		r2(A)			R2(C)		w2(D)			
T3				w3(C)						
T4						r4(B)		w4(E)		
T5									r5(D)	w5(E)

Step1
Find conflicts
(RW, WW, WR)

Step2
Build Conflict graph
Acyclic?



Acyclic
⇒ Conflict serializable!
⇒ Serializable

Step3
Example serial schedule
Conflict Equiv to S1

S2

T3	T1	T1	T4	T4	T2	T2	T2	T5	T5
w3(C)	w1(A)	w1(B)	r4(B)	w4(E)	r2(A)	r2(C)	w2(D)	r5(D)	w5(E)

Summary

- Concurrency achieved by **interleaving TXNs** such that **isolation** & **consistency** are maintained
  - We formalized a notion of **serializability** that captured such a “good” interleaving schedule

- We defined **conflict serializability**

A few parting observations

- Often, we can construct many conflict serializable schedules
  - Increased flexibility/degrees of freedom are great!
  - We can choose the best performing among the serial schedules

- How many transactions should we schedule at once?
  - The more transactions we schedule in a batch --> higher concurrency and throughput, more possible schedules and degrees of freedom
  - But…
    - Higher latency
    - The scheduler takes longer and becomes more complex